INFORME TECNICO

SISTEMA DE GESTION DE TORNEO DE FUTBOL

Autor: Samuel Polanco Martínez

Versión: 1.0.0

Fecha: enero 2026

1. INTRODUCCION

Este documento describe la estructura técnica, arquitectura y decisiones de diseño tomadas durante el desarrollo del Sistema de Gestión de Torneo de Fútbol. El proyecto fue desarrollado utilizando Python 3.13, PySide6 para la interfaz gráfica y SQLite3 como motor de la base de datos.

2. OBJETIVOS DEL PROYECTO

2.1. Objetivo General

Desarrollar una aplicación de escritorio que permita gestionar de forma completa un torneo de fútbol por eliminatorias, incluyendo equipos, jugadores, árbitros, calendario de partidos y estadísticas.

2.2. Objetivos Específicos

- Implementar un sistema de gestión de equipos con información personalizable
- Registrar participantes (jugadores y árbitros) con sus estadísticas
- Programar partidos organizados por eliminatorias
- Actualizar resultados y estadísticas de forma automática
- Generar clasificaciones y reportes estadísticos
- Proporcionar una interfaz intuitiva y fácil de usar

3. ARQUITECTURA DEL PROYECTO

3.1. Patrón de Diseño: MVC (Modelo-Vista-Controlador)

Se eligió el patrón MVC por las siguientes razones:

- Separación clara de responsabilidades
- Facilidad de mantenimiento y escalabilidad
- Reutilización de código

- Testeo independiente de cada capa

MODELO (models/)

Responsabilidad: Gestión de datos y lógica de negocio relacionada con la base de datos

Archivos:

- database.py: Conexión a la base de datos y creación de tablas
- equipo.py: Operaciones CRUD para equipos
- participante.py: Operaciones CRUD para participantes
- partido.py: Operaciones CRUD para partidos
- estadistica_partido.py: Gestión de estadísticas individuales por partido

VISTA (views/)

Responsabilidad: Presentación de la interfaz gráfica al usuario

Archivos:

- mainwindow.py: Código generado automáticamente desde TorneoFutbol.ui
- TorneoFutbol.ui: Diseño de interfaz creado con Qt Designer

CONTROLADOR (controllers/)

Responsabilidad: Lógica de negocio y coordinación entre modelo y vista

Archivos:

- main_controller.py: Controlador principal que gestiona toda la lógica de la aplicación

3.2. Estructura de Directorios

```
Torneo_Futbol/
 ├── main.py                # Punto de entrada de la aplicación
 ├── TorneoFutbol.ui        # Diseño de interfaz (Qt Designer)
 ├── torneoFutbol_sqlite.db # Base de datos SQLite
 ├── README.md              # Documentación del proyecto
 ├── MANUAL_USUARIO.txt     # Manual de usuario
 ├── INFORME_TECNICO.txt    # Este documento
 |
```

```
├── models/                # Capa de Modelo (Datos)
│   ├── __init__.py
│   ├── database.py        # Gestión de conexión y tablas
│   ├── equipo.py          # Modelo Equipo
│   ├── participante.py    # Modelo Participante
│   ├── partido.py         # Modelo Partido
│   └── estadistica_partido.py # Modelo EstadisticaPartido
│
├── views/                 # Capa de Vista (Presentación)
│   ├── __init__.py
│   └── mainwindow.py      # Código UI generado automáticamente
│
├── controllers/           # Capa de Controlador (Lógica)
│   ├── __init__.py
│   └── main_controller.py # Controlador principal
│
└── resources/             # Recursos adicionales
    ├── __init__.py
    └── style.qss          # Estilos CSS para la interfaz
```

4. TECNOLOGIAS UTILIZADAS

4.1. Python 3.13

Razones de elección:

- Lenguaje de alto nivel con sintaxis clara y legible
- Amplio ecosistema de librerías
- Excelente soporte para desarrollo de aplicaciones de escritorio
- Facilidad para el manejo de bases de datos

4.2. PySide6 (Qt for Python)

Razones de elección:

- Framework robusto y maduro para interfaces gráficas

- Multiplataforma (Windows, Linux, macOS)
- Qt Designer permite diseño visual de interfaces
- Amplia documentación y comunidad activa
- Componentes ricos y personalizables
- Manejo eficiente de eventos

4.3. SQLite3

Razones de elección:

- Base de datos ligera sin necesidad de servidor
- Integrada por defecto en Python
- Ideal para aplicaciones de escritorio
- Transaccional y confiable
- Bajo consumo de recursos
- Fácil portabilidad (un solo archivo)

4.4. Qt Designer

Razones de elección:

- Diseño visual de interfaces WYSIWYG
- Generación automática de código Python
- Facilita el desarrollo rápido de interfaces
- Mantenimiento independiente de la lógica

5. BASE DE DATOS

5.1. Diseño del Esquema

TABLA: equipos

Almacena la información de los equipos participantes

Campos:

- id (INTEGER PRIMARY KEY AUTOINCREMENT): Identificador único
- nombre (TEXT NOT NULL): Nombre del equipo
- curso (TEXT): Curso al que pertenece (ej: 1ºA, 2ºB)
- color_camiseta (TEXT): Color de la camiseta del equipo

- emblema (TEXT): Ruta del archivo de imagen del emblema

- informacion_adicional (TEXT): Información extra del equipo

TABLA: participantes

Almacena jugadores y árbitros

Campos:

- id (INTEGER PRIMARY KEY AUTOINCREMENT): Identificador único

- nombre (TEXT NOT NULL): Nombre completo

- fecha_nacimiento (TEXT): Fecha de nacimiento

- curso (TEXT): Curso del participante

- es_jugador (BOOLEAN DEFAULT 0): Indica si es jugador

- es_arbitro (BOOLEAN DEFAULT 0): Indica si es árbitro

- posicion (TEXT): Posición del jugador (Portero, Defensa, Centrocampista, Delantero)

- equipo_id (INTEGER): Clave foránea a tabla equipos

- goles (INTEGER DEFAULT 0): Total de goles anotados

- t_amarillas (INTEGER DEFAULT 0): Total de tarjetas amarillas

- t_rojas (INTEGER DEFAULT 0): Total de tarjetas rojas

- FOREIGN KEY (equipo_id) REFERENCES equipos(id)

TABLA: partidos

Almacena los partidos del torneo

Campos:

- id (INTEGER PRIMARY KEY AUTOINCREMENT): Identificador único

- equipo_local_id (INTEGER NOT NULL): Equipo local

- equipo_visitante_id (INTEGER NOT NULL): Equipo visitante

- fecha (TEXT): Fecha del partido

- hora (TEXT): Hora del partido

- arbitro_id (INTEGER): Árbitro asignado

- goles_local (INTEGER): Goles del equipo local

- goles_visitante (INTEGER): Goles del equipo visitante

- jugado (BOOLEAN DEFAULT 0): Indica si el partido ya se jugó

- eliminatoria (TEXT): Fase del torneo (Octavos, Cuartos, etc.)

- lugar (TEXT): Lugar donde se juega el partido

- FOREIGN KEY (equipo_local_id) REFERENCES equipos(id)

- FOREIGN KEY (equipo_visitante_id) REFERENCES equipos(id)

- FOREIGN KEY (arbitro_id) REFERENCES participantes(id)

TABLA: estadisticas_partido

Almacena estadísticas individuales de cada partido

Campos:

- id (INTEGER PRIMARY KEY AUTOINCREMENT): Identificador único

- partido_id (INTEGER NOT NULL): Partido asociado

- participante_id (INTEGER NOT NULL): Jugador asociado

- goles (INTEGER DEFAULT 0): Goles anotados en el partido

- tarjetas_amarillas (INTEGER DEFAULT 0): Tarjetas amarillas en el partido

- tarjetas_rojas (INTEGER DEFAULT 0): Tarjetas rojas en el partido

- FOREIGN KEY (partido_id) REFERENCES partidos(id)

- FOREIGN KEY (participante_id) REFERENCES participantes(id)

5.2. Índices

Se crearon índices para optimizar las consultas frecuentes:

- idx_participantes_equipo: En participantes(equipo_id)

- idx_partidos_equipos: En partidos(equipo_local_id, equipo_visitante_id)

- idx_estadisticas: En estadisticas_partido(partido_id, participante_id)

5.3. Decisiones de Diseño

Uso de BOOLEAN como INTEGER:

SQLite no tiene tipo BOOLEAN nativo, se utilizan valores 0 (false) y 1 (true) almacenados como INTEGER.

Almacenamiento de fechas como TEXT:

Las fechas se almacenan en formato 'YYYY-MM-DD' como TEXT para facilitar las consultas y la compatibilidad.

Normalización:

Se aplicó tercera forma normal para evitar redundancia de datos y mantener integridad referencial.

6. DECISIONES DE DISEÑO

6.1. Interfaz de Usuario

StackedWidget para navegación:

Se utilizó QStackedWidget para gestionar múltiples páginas en una sola ventana, proporcionando una navegación fluida sin abrir ventanas nuevas.

QTreeWidget para partidos:

Los partidos se organizan jerárquicamente en un árbol (Eliminatoria → Fecha → Partido), facilitando la visualización de la estructura del torneo.

Formularios dinámicos:

Los formularios se habilitan/deshabilitan según el contexto para prevenir errores del usuario.

Colores para estados:

- Verde: Partido jugado
- Naranja: Partido pendiente

Mejora la comprensión visual del estado del torneo.

6.2. Validaciones

Validación de equipos diferentes:

No se permite que un equipo juegue contra sí mismo. Se valida al crear y editar partidos.

Validación de fases secuenciales:

No se pueden programar partidos de una fase si la anterior no está completa. Esto garantiza la lógica del torneo por eliminatorias.

Filtrado de equipos ganadores:

Solo los equipos que han ganado su partido pueden ser seleccionados para la siguiente fase.

6.3. Gestión de Dato

Actualización automática de estadísticas:

Al registrar resultados de un partido, las estadísticas globales de jugadores se actualizan automáticamente mediante triggers o lógica en el controlador.

Integridad referencial:

Se utilizan claves foráneas para mantener la consistencia de los datos entre tablas.

Transacciones:

Las operaciones que modifican múltiples tablas se ejecutan en transacciones para garantizar atomicidad.

7. FUNCIONALIDADES CLAVE

7.1. Gestión de Equipos

- CRUD completo (Crear, Leer, Actualizar, Eliminar)
- Selección de color mediante QColorDialog
- Carga de emblemas desde archivos de imagen
- Visualización de jugadores asociados

7.2. Gestión de Participantes

- Registro de jugadores con posición y equipo
- Registro de árbitros
- Un participante puede ser jugador y árbitro simultáneamente
- Estadísticas individuales (goles, tarjetas)

7.3. Gestión del Calendario

- Programación de partidos por eliminatorias
- Visualización en árbol jerárquico
- Edición y eliminación de partidos

- Validación de fases secuenciales

- Botón de reinicio de torneo

7.4. Actualización de Resultados

- Registro de goles por equipo

- Registro de estadísticas individuales por jugador

- Marcado automático de partido como jugado

- Actualización automática de estadísticas globales

7.5. Clasificación y Eliminatorias

- Cuadro de eliminatorias con resultados

- Clasificación general de equipos

- Ranking de goleadores

- Tabla de tarjetas

8. PATRONES Y BUENAS PRACTICAS

8.1. Patrones Aplicados

Singleton en Database:

La conexión a la base de datos se gestiona como Singleton para evitar múltiples conexiones.

Factory Method en Models:

Los modelos utilizan métodos estáticos que actúan como factory methods para crear instancias.

Observer en Signals/Slots:

PySide6 implementa el patrón Observer mediante signals y slots para la comunicación entre componentes.

8.2. Principios SOLID

Single Responsibility:

Cada clase tiene una única responsabilidad bien definida.

Open/Closed:

Las clases están abiertas a extensión pero cerradas a modificación.

Dependency Inversion:

El controlador depende de abstracciones (modelos) no de implementaciones concretas.

8.3. Gestión de Errores

Try-Except en operaciones críticas:

Todas las operaciones de base de datos están protegidas con manejo de excepciones.

Mensajes informativos:

Se utilizan QMessageBox para informar al usuario de errores y confirmaciones.

Logging:

Se implementa logging en consola para depuración durante el desarrollo.

9. OPTIMIZACIONES

9.1. Rendimiento

Uso de índices:

Los índices en la base de datos aceleran las consultas frecuentes.

Carga diferida:

Los datos se cargan solo cuando son necesarios, no al iniciar la aplicación.

Reutilización de queries:

Se preparan queries con placeholders para reutilizar y prevenir SQL injection.

9.2. Experiencia de Usuario

Tooltips informativos:

Se añadieron tooltips en campos importantes para guiar al usuario.

Autocompletado:

Los combos de equipos y participantes se actualizan automáticamente.

Feedback visual:

Cambios de color y símbolos indican el estado de los elementos.

10. SEGURIDAD

10.1. SQL Injection

Consultas parametrizadas:

Todas las consultas utilizan placeholders (?) para prevenir inyección SQL.

10.2. Validación de Entrada

Validación de tipos:

Se valida que los datos ingresados sean del tipo correcto antes de guardar.

Validación de rangos:

Los valores numéricos se validan para estar en rangos lógicos.

11. TESTING Y DEPURACION

11.1. Estrategia de Testing

Testing manual:

Se realizaron pruebas manuales exhaustivas de todas las funcionalidades.

Casos de prueba:

- Creación de equipos con y sin emblema
- Registro de jugadores y árbitros
- Programación de partidos secuenciales
- Actualización de resultados
- Validación de restricciones

11.2. Depuración

Print statements:

Se utilizaron prints estratégicos para depurar durante el desarrollo.

Verificación de base de datos:

Se utilizó SQLite Browser para verificar la integridad de los datos.

12. LIMITACIONES Y MEJORAS FUTURAS

12.1. Limitaciones Actuales

- No hay sistema de autenticación de usuarios
- No se pueden deshacer operaciones (sin sistema de undo)
- Los emblemas se almacenan como rutas, no como BLOB
- No hay exportación de reportes a PDF
- Interfaz en un solo idioma (español)

12.2. Mejoras Propuestas

Autenticación:

Implementar sistema de usuarios con roles (administrador, operador).

Historial de cambios:

Agregar auditoría de todas las modificaciones.

Reportes avanzados:

Generar reportes en PDF con gráficos estadísticos.

Modo oscuro:

Implementar tema oscuro para la interfaz.

Internacionalización:

Soporte para múltiples idiomas.

Sincronización en la nube:

Opción de sincronizar datos con servidor remoto.

Notificaciones:

Sistema de recordatorios para partidos próximos.

13. MANTENIMIENTO

13.1. Actualización de la Base de Datos

Para agregar nuevos campos:

1. Modificar el esquema en database.py
2. Ejecutar migración manual o recrear la base de datos
3. Actualizar los modelos correspondientes

13.2. Actualización de la Interfaz

Para modificar la interfaz:

1. Editar TorneoFutbol.ui con Qt Designer
2. Regenerar mainwindow.py con: pyside6-uic TorneoFutbol.ui -o views/mainwindow.py
3. Nunca editar mainwindow.py manualmente (se sobrescribe)

13.3. Versionado

Se recomienda usar control de versiones (Git) para:

- Rastrear cambios en el código
- Facilitar colaboración
- Permitir rollback en caso de errores

14. CONCLUSIONES

El proyecto representa una solución completa y profesional para la gestión de torneos deportivos. La arquitectura modular facilita futuras extensiones y el código está documentado y organizado siguiendo estándares de la industria.